

License

This **graphical library** is distributed under the **Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)** license.

You are free to use, modify, and distribute this code for personal, educational, and **non-commercial purposes**, provided that proper authorship credit is given. Use of this library in any **commercial projects is strictly prohibited**. For inquiries regarding the acquisition of a **commercial license**, please contact: kboruzdov@mail.ru, tel. +7 965-363-45-29.

Intellectual Property Protection and Proprietary Algorithms

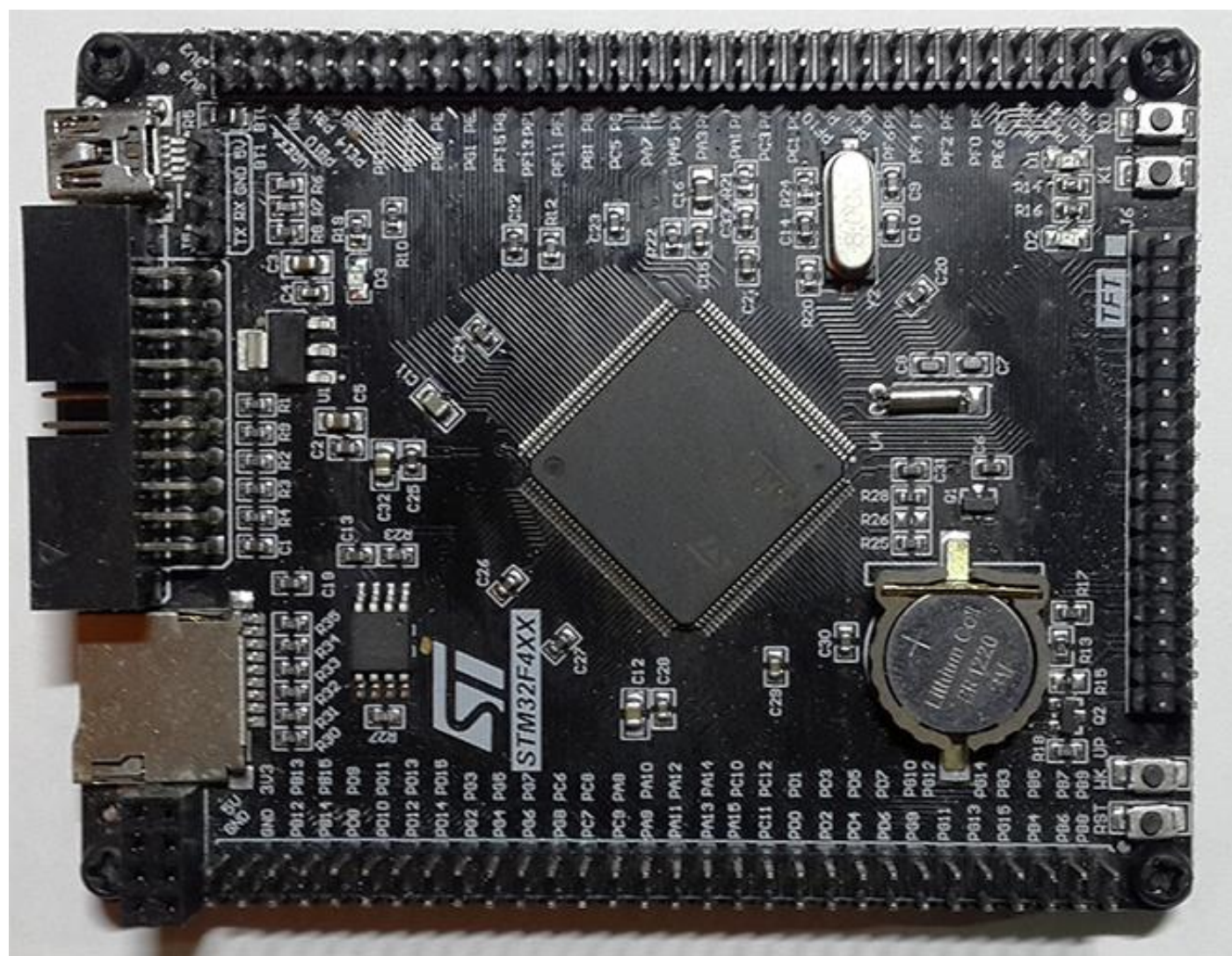
The mathematical solutions and algorithms implemented within the library's graphics core are the exclusive intellectual property of the author. Specifically, the proprietary algorithm for on-the-fly rendering and gradient filling of circular/three-surface objects, executed purely via integer math (without utilizing a frame buffer), represents closed, protected know-how.

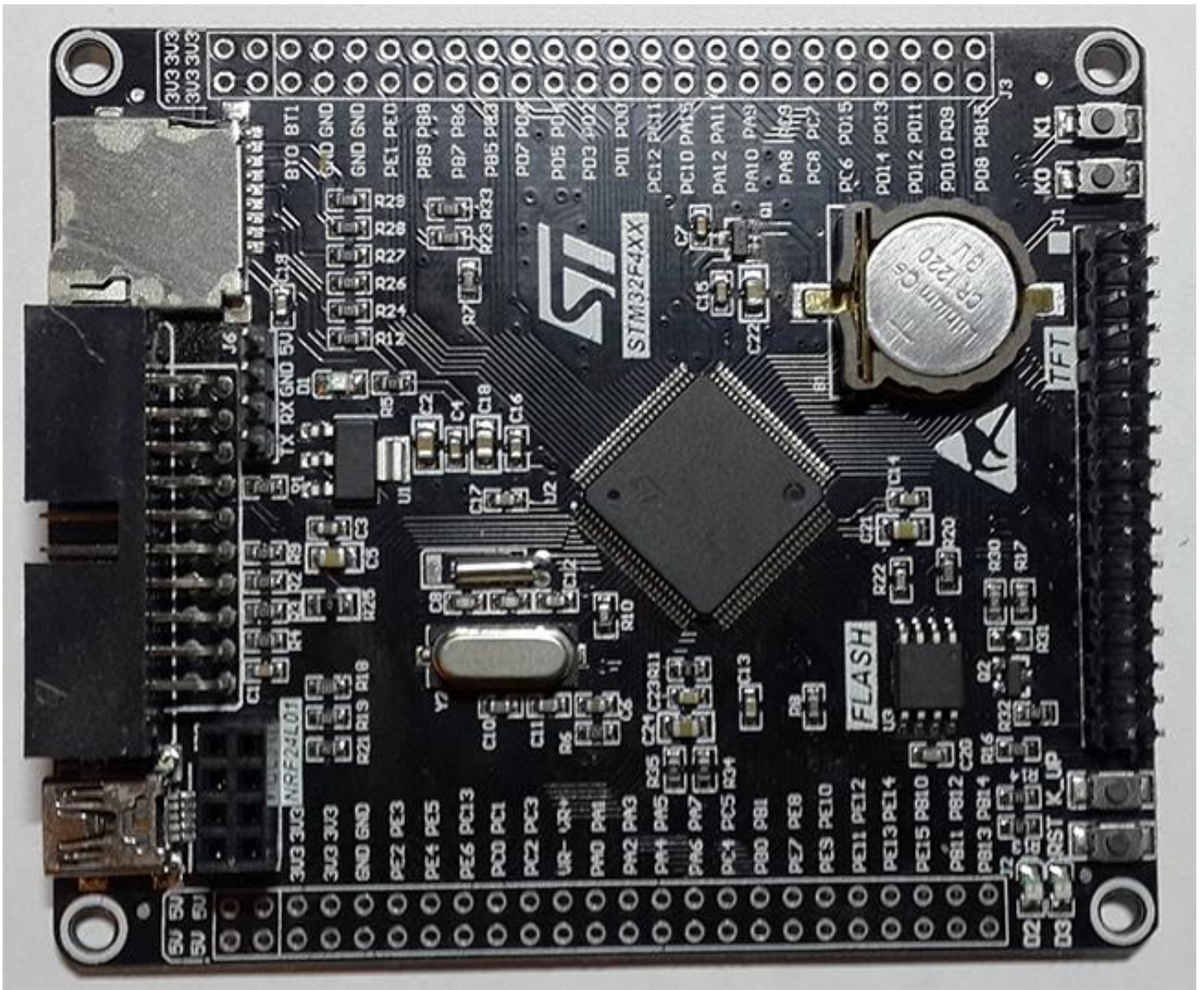
To protect copyright and prevent unauthorized replication of these unique, low-level engineering solutions, the graphics core is distributed strictly as precompiled, closed-source binary modules (static library files in `.a` and `.lib` formats).

Under no circumstances shall the source code of the mathematical core (including high-speed line rendering, **GRAM** gradient calculation, and three-surface object algorithms) be disclosed or transferred, regardless of the commercial license type acquired. However, the developer receives full access to the source code of the **GUI** interface layer (`KVB_LCD.h`), display drivers, and demonstration projects, ensuring complete flexibility for integration and custom user interface development.

Cross-Platform Compatibility

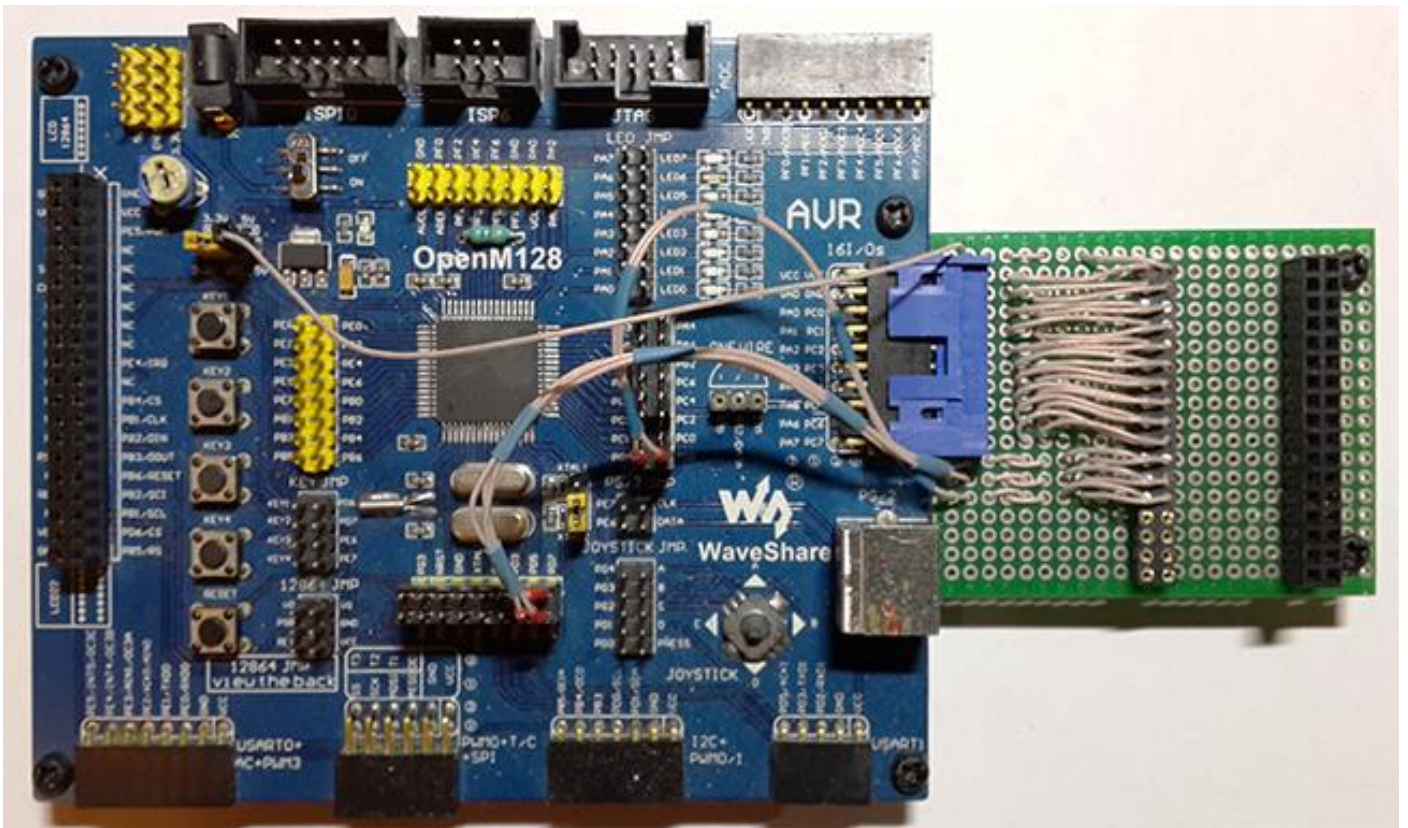
The library is written **from scratch in pure C** within **GCC** and **AC6** toolchains, entirely free of third-party code, and utilizes **integer-only math**. Currently, the library supports two





To view the test presentation code for **ATmega128**, the following I/O ports are used:

1. **PORTC (D0-D7):** Low-byte data bus of the LCD panel.
2. **PORTA (D8-D15):** High-byte data bus of the LCD panel.
3. **PORTD pin 4 (CS):** Chip Select signal transmission to the LCD panel.
4. **PORTD pin 5 (RD):** Read signal transmission to the LCD panel.
5. **PORTD pin 6 (WR):** Write signal transmission to the LCD panel.
6. **PORTD pin 7 (RS):** Register Select (Command/Data) signal transmission.
7. **PORTG pin 0 (RST):** Reset signal transmission to the LCD panel.
8. **PORTG pin 1 (BL):** Backlight signal transmission to the LCD panel.



LCD Panels

Currently, drivers have been written for **five LCD panels**:

1. **ILI9325** with a resolution of **320 x 240**.
2. **ILI9341** with a resolution of **320 x 240**.
3. **ILI9481** with a resolution of **480 x 320**.
4. **ILI9484** with a resolution of **480 x 320**.
5. **OTM8009A** with a resolution of **800 x 480**.

The pinouts of the board and panel connectors are attached in the "**FSMC LCD**" folder as **.xlsx documents**. It is quite possible that your **LCD** panel connectors may differ, but the board connectors must match.

Unique Functionality and Proprietary Algorithms

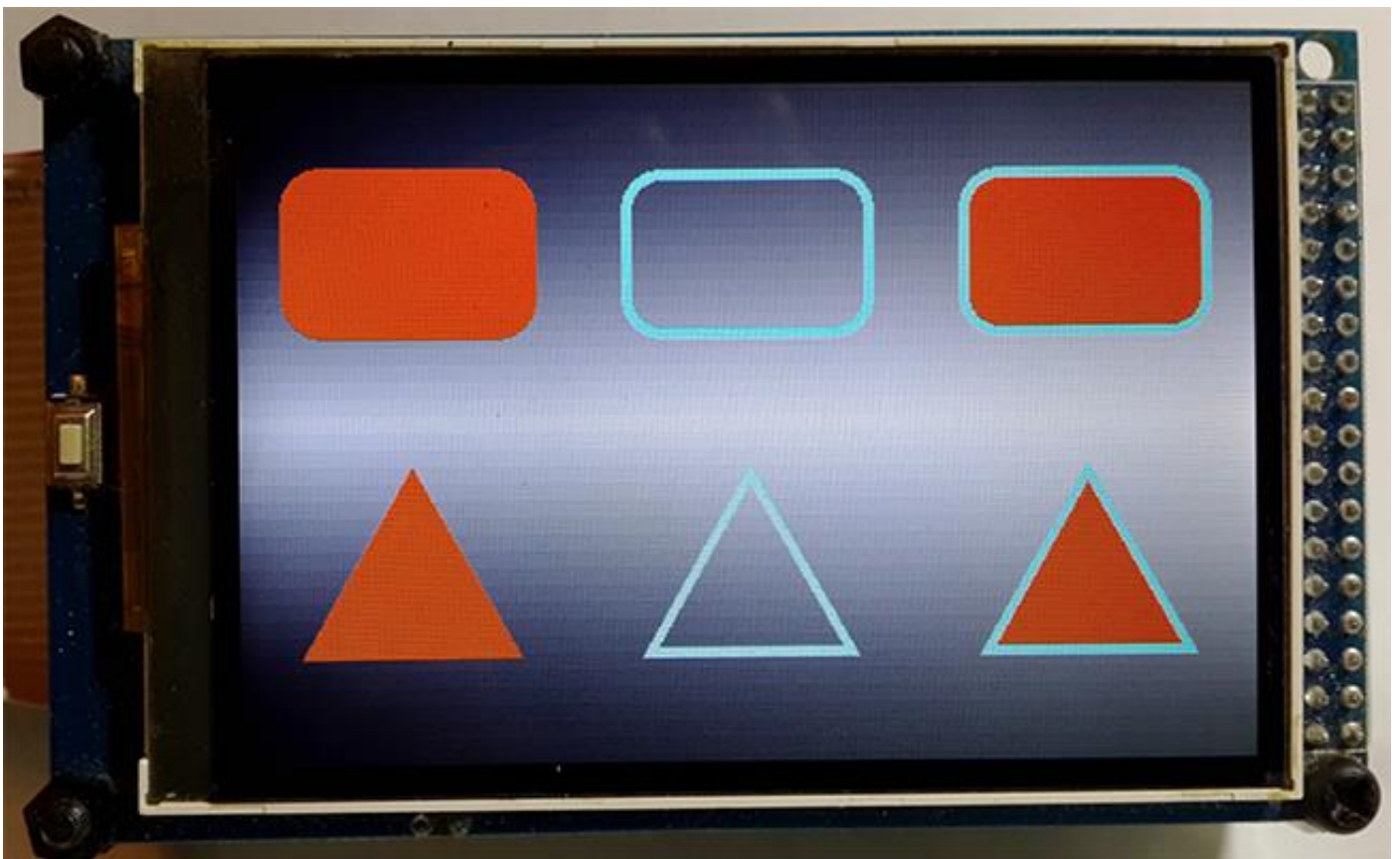
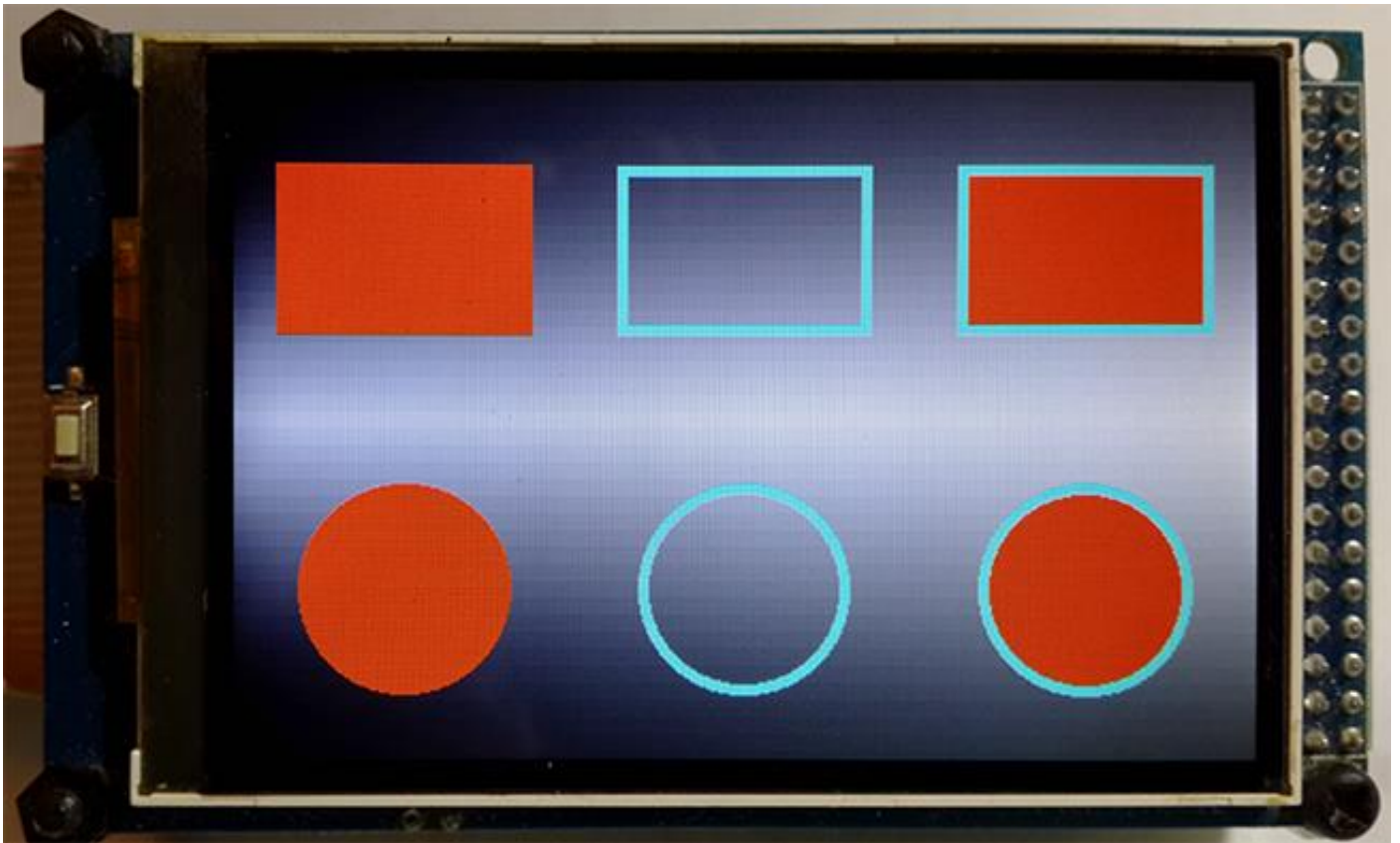


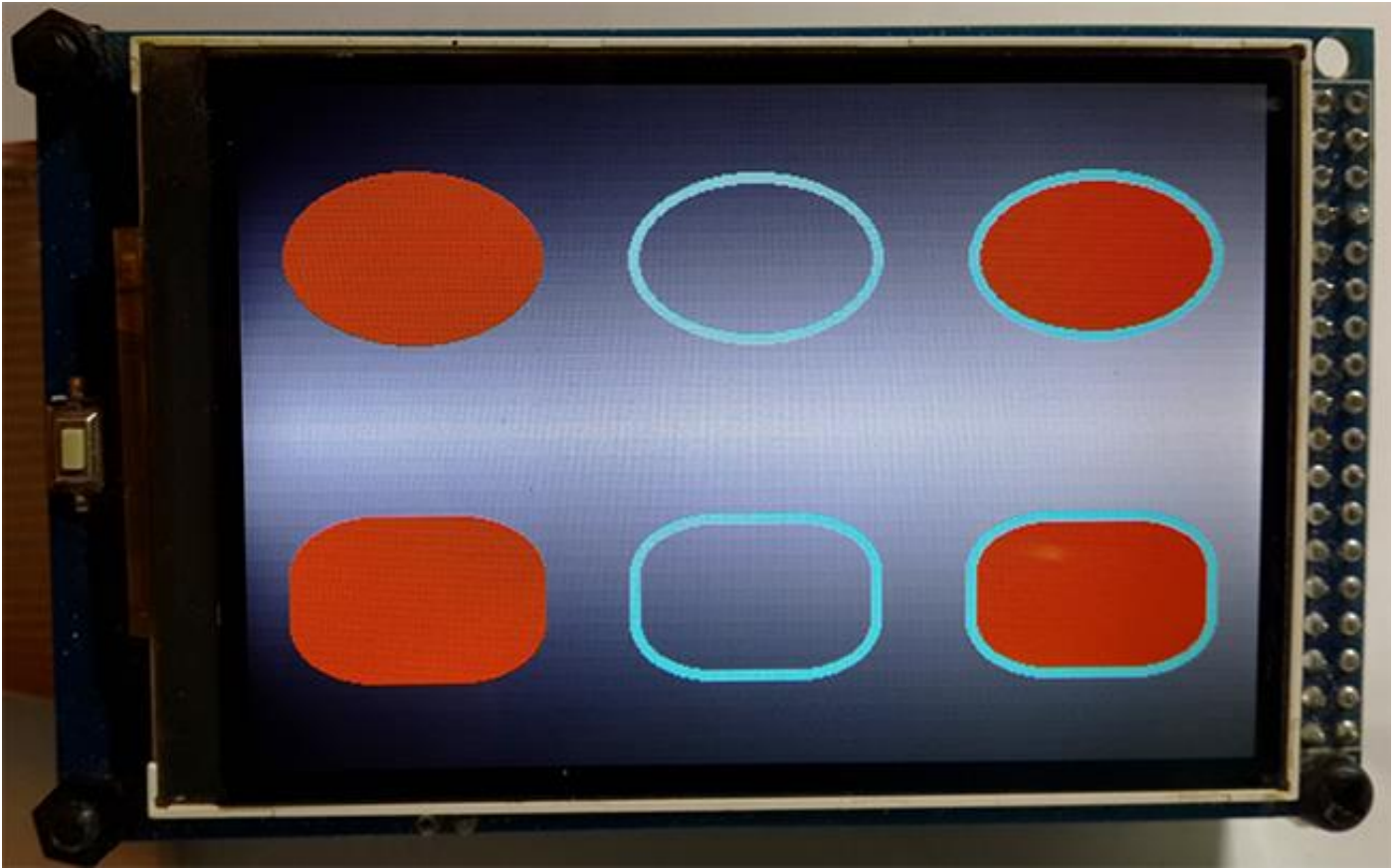


Beyond cross-platform compatibility, the primary objective when designing this library was achieving the **maximum possible rendering speed** for graphical objects. Unlike conventional libraries that rely on standard pixel-by-pixel rendering, this project implements **proprietary optimized algorithms** tailored specifically to the architecture of display **Graphics RAM (GRAM)**:

1. **High-Speed Line Rendering:** Utilizes a **proprietary algorithm** that calculates continuous straight segments within diagonal lines. This minimizes the number of **GRAM address switching commands** sent to the **LCD panel** and enables rapid **block filling (burst fill)**, which drastically multiplies rendering speed and **FPS**.
 2. **Gradient Color Rendering:** Implements a **proprietary algorithm** that calculates and renders vertical and horizontal gradients (with either **linear or centered orientation**) by leveraging the **hardware acceleration capabilities** of the **LCD panel's GRAM matrix**.
 3. **Multifunctional Three-Surface Object:** Employs a **proprietary algorithm** that computes filled gradient circles **on-the-fly** using integer arithmetic, completely **bypassing the need for framebuffers**. This object concurrently calculates and renders **three rounded, circular, or rectangular surfaces** with gradients in different directions in real time. All three surfaces are **completely independent** and can be redrawn separately. This allows the system to highlight an object selection by **repainting only the single outer surface**, without wasting **CPU cycles** on re-rendering the remaining surfaces.
-

Graphical Primitives





Features **three display modes** and an **adjustable border**. Graphical primitives contain two independent surfaces and support three rendering modes: **1 = entire primitive**, **2 = adjustable border only**, and **3 = inner surface only**. This allows the UI to select or highlight an object on the LCD screen by **changing its border color** without spending time re-rendering its inner body. Triangle primitives (both equilateral and isosceles) support **four vertex orientations**: **1 = left**, **2 = right**, **3 = up**, and **4 = down**.

Advanced Text Rendering Subsystem

Текст с тенью

Съешь ещё этих мягких французских
булок, да выпей же чаю. Съешь ещё этих
мягких французских булок, да выпей же
чаю. Съешь ещё этих мягких французских
булок, да выпей же чаю. Выводим в
строке вещественное число с добавлением
нулей. -0002350,651 Съешь ещё этих
мягких французских булок, да выпей же
чаю. Съешь ещё этих мягких французских
булок, да выпей же чаю.

чаю. Выводим
число с доб.



The engine features a powerful, high-speed **text processing engine** that automatically aligns strings and renders complex visual effects **on-the-fly without a framebuffer**:

1. **Baseline Alignment:** All fonts align to a **single horizontal guide** (the bottom shelf of characters). This makes it possible to assemble a single line out of words of **varying sizes and colors** while ensuring perfect text alignment.
2. **Automatic Word Wrapping:** Long continuous text blocks can consist of a mix of **string, integer, and float variables** of different font sizes and colors. The engine automatically wraps text by words to the next line if it exceeds the **text block width boundary**.
3. **Tabulation Support:** Full support for tab stops.
4. **Font Kerning:** Adjustable **inter-character spacing** with automatic length correction for spaces and tabs.
5. **Line Spacing Control:** Spacing can be set to both **positive and negative** values. Negative spacing is highly effective when rendering long text blocks using **ALL CAPS**. Positive spacing enhances text readability.
6. **Font Scaling Factor:** Support for font magnification scaling.
7. **Text Alignment:** Alignment can be set to **left-aligned, right-aligned, or centered**.
8. **Shadow Effect:** Character shadows feature four configurations: **1 = bottom-right, 2 = bottom-left, 3 = top-right, and 4 = top-left**. This creates an **embossed (raised) or engraved (recessed) font effect**, significantly enhancing text readability.

9. **Character Outlining:** Renders a crisp **circular outline** around each character to ensure readability against **any background color**. The system can also be set to display **only the outline** without the inner character fill.
 10. **Gradient Text:** Delivers smooth **horizontal or vertical color transitions** via linear or centered line fills, which can be combined with shadows or outlines.
-

Delivery Format and Source Code Availability

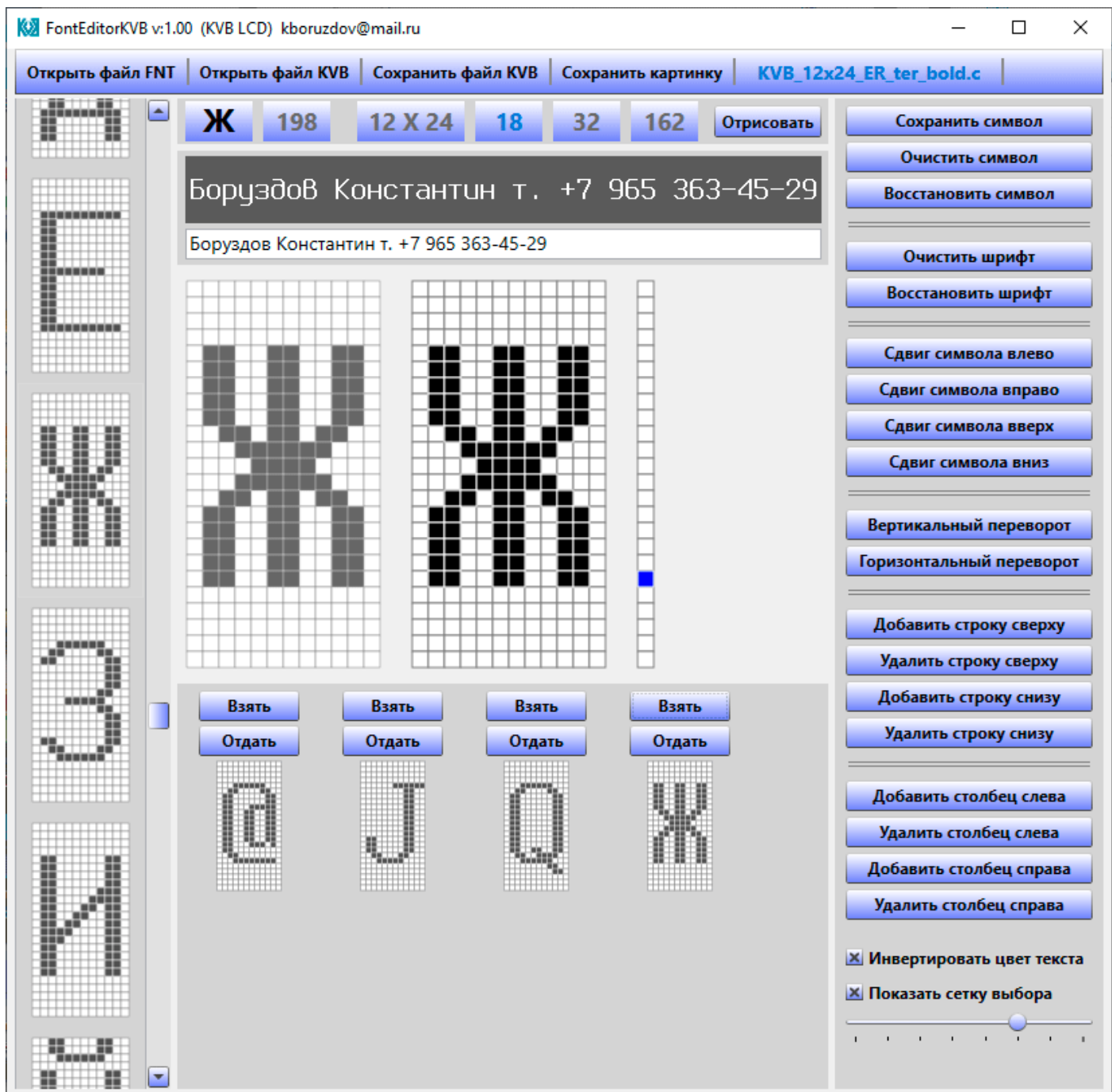
The **core graphical engine** of the library is supplied as **pre-compiled closed static libraries** to protect the proprietary rendering algorithms:

- For **STM32F407**: Static library files in **.a** format (for **STM32CubeIDE**) and **.lib** format (for Keil MDK).
- For **ATmega128**: Static library files in **.a** format (for **Atmel Studio 7.0**).

What remains fully open to the developer:

- The **main GUI interface header file** — **KVB_LCD.h**. It is meticulously and extensively **commented in Russian**, providing descriptions for all functions, parameters, font effects, and graphical objects.
 - All **demonstration and presentation code**, which includes detailed Russian comments for a **fast start**.
-

Font Editor (Windows IDE)



To facilitate the creation and customization of screen fonts, a dedicated **standalone Windows application** has been developed to automate the entire graphics resource preparation workflow:

1. **DOS Font Import (.fnt):** Allows rapid loading of classic raster fonts from the **DOS era** for subsequent adaptation to microcontroller displays.
2. **Pixel-by-Pixel Editing:** Provides an intuitive graphical interface for **manual drawing**, pixel-grid character modification, and font metrics adjustment.
3. **Interactive Rendering:** Displays a given text string **on-the-fly in real time**. You can see instantly how modifying a single pixel in a character affects the rendering of the whole line.

4. **Export to C Format:** Generates production-ready .c files utilizing an **ultra-compact sequential data packing method** (a 9x16 character occupies **just 18 bytes**). These arrays are fully compatible with both PROGMEM in the **ATmega128 Flash memory** and standard const arrays in the **STM32F407**.
5. **Proprietary Font Pack:** Using this editor, **23 unique Russian-English fonts** have been created (**5** base fonts are included in the presentation for demonstration; the full set and editor are available upon request).

Note: The demo version of the editor disables the C-format export feature; the fully unlocked editor is available upon request.